
Autonomous Fixed-Wing UAV

Flight Dynamics, Robust Control and Optimisation Simulator

A step-by-step build guide

From an empty folder to a statistically validated result

Every phase is explained three times: once simply, once in mathematics, and once as the code you actually have to write. Each phase ends with a gate — a test that must pass before you are allowed to move on.

Companion to	Project Definition v0.1
Scope	Flight dynamics, control, optimisation, uncertainty
Out of scope	Weapons, targeting, engagement, payload delivery
Language	Python (NumPy / SciPy), C++ only after profiling
Version	Build Guide v1.0

Contents

How to read this guide	4
0 Phase 0 — Freeze the rules	6
0.1 The maths: choose and write these down	6
0.2 What you code	6
1 Phase 1 — Make it move (6-DOF rigid body)	8
1.1 The maths	8
1.2 What you code	9
1.3 The gate tests, in detail	9
2 Phase 2 — Give it wings (aerodynamics and propulsion)	10
2.1 The maths	10
2.2 What you code	11
2.3 The gate tests	11
3 Phase 3 — Find level flight and the linear model	12
3.1 The maths	12
3.2 What you code	13
3.3 The gate tests	13
4 Phase 4 — First autopilot (cascaded PID)	14
4.1 The maths	14
4.2 What you code	14
4.3 The gate tests	15
5 Phase 5 — Make the world unfair	16
5.1 The maths	16
5.2 What you code	17
5.3 The gate tests	17
6 Phase 6 — The better autopilot	18
6.1 The maths: nonlinear dynamic inversion	18
6.2 What you code	19
6.3 Fair comparison rules	19
7 Phase 7 — Auto-tuning (optimisation)	20
7.1 The maths	20
7.2 What you code	21
7.3 The gate tests	21
8 Phase 8 — Learn the leftovers (optional but impressive)	22
8.1 The maths	22
8.2 What you code	22
8.3 The gate tests	23
9 Phase 9 — Ten thousand flights	24
9.1 The maths	24
9.2 What you code	25
9.3 The gate tests	25

10 Phase 10 — Report and demonstration website	26
10.1 The export contract	26
10.2 What the website shows, in order	26
10.3 Report structure	26
A Appendix A — Suggested repository layout	28
B Appendix B — Master gate checklist	28
C Appendix C — The five research questions, and where each is answered	29

How to read this guide

The whole project, explained to a ten-year-old

In plain words. Imagine you want to teach a paper aeroplane to fly by itself. You cannot just throw it and hope. So instead you build a *pretend aeroplane inside a computer*. The computer knows where the plane is, which way it is pointing, and how fast it is going. Every few thousandths of a second it asks: “what forces are pushing on this plane right now?” — then it moves the plane a tiny bit. Do that a hundred thousand times and you have a flight.

Once the pretend plane flies properly, you write a second program — the *autopilot* — whose only job is to wiggle the little flaps on the wings and tail so the plane goes where you tell it. That is the easy-sounding part that is actually hard, because the real world is annoying: the wind pushes, the sensors lie a bit, the flaps move slowly, and you never know the plane’s exact weight.

So the last step is the clever one. Instead of testing your autopilot on *one* pretend plane, you build *ten thousand slightly different* pretend planes — some heavier, some with weaker wings, some flying through gusts — and you fly all of them. At the end you can say something no one-plot project can say:

“My autopilot flew properly in 97.3% of ten thousand different worlds. The old one only managed 78.1%.”

That sentence is the whole project. Everything else exists to earn the right to say it.

The structure of every phase

Each phase below has exactly four parts:

1. **In plain words** — the intuition, no symbols.
2. **The maths** — the equations you are implementing, written out.
3. **What you code** — the actual files and function signatures.
4. **The gate** — the test that must pass before you move on.

Trap. The gates are the entire point of this document. A flight simulator can contain 300 correct equations and one wrong minus sign, and it will still run, still produce plots, and still be completely worthless. Every hour you spend on gates saves you a week of debugging a controller that was never broken.

Notation used throughout

Symbol	Meaning
$\mathbf{p}^n = [p_N, p_E, p_D]^T$	Position in the North-East-Down (NED) inertial frame, m
$\mathbf{v}^b = [u, v, w]^T$	Velocity in the body frame, m/s
$\boldsymbol{\omega}^b = [p, q, r]^T$	Angular rates: roll, pitch, yaw, rad/s
$\mathbf{q} = [q_0, q_1, q_2, q_3]^T$	Attitude quaternion, scalar-first
ϕ, θ, ψ	Euler angles: roll, pitch, yaw (for plots only)
V	True airspeed, m/s
α, β	Angle of attack, sideslip angle, rad
$\delta_a, \delta_e, \delta_r, \delta_t$	Aileron, elevator, rudder, throttle

Symbol	Meaning
$m, \mathbf{I}$	Mass (kg), inertia tensor (kg m^2)
$\rho, S, b, \bar{c}$	Air density, wing area, span, mean chord
C_L, C_D, C_Y	Lift, drag, sideforce coefficients
C_l, C_m, C_n	Roll, pitch, yaw moment coefficients
$\mathbf{R}_b^n$	Rotation matrix, body frame to NED frame
$\boldsymbol{\theta}_u$	Vector of <i>uncertain</i> parameters (Monte Carlo)
$\mathbf{k}$	Vector of <i>tunable</i> controller gains

The map: what depends on what

Phase	Name	You cannot start it until...
0	Freeze the rules	— (start here)
1	Make it move	0 is written down and agreed
2	Give it wings	1 conserves momentum in free fall
3	Find level flight	2 produces sensible forces
4	First autopilot	3 gives correct aircraft modes
5	Make the world unfair	4 works on the perfect aircraft
6	Better autopilot	5 has degraded phase 4 measurably
7	Auto-tuning	6 beats 4 by hand-tuning
8	Learn the leftovers	7 has a stable, tuned controller
9	Ten thousand flights	8 (or skip 8) and all gates passed
10	Report and website	9 produced a results database

0 Phase 0 — Freeze the rules

In plain words. Before two people play a board game, they agree the rules. Which way is forward? Does a positive number mean up or down? If you skip this and just start playing, you will argue for three hours about whether someone cheated. In this project, that argument is called “debugging”, and it can eat a month.

So phase 0 has *no code at all*. You write a short document that says exactly what every number in your program means. Then you never change it.

0.1 The maths: choose and write these down

1. Frames. Use North-East-Down (NED) for the inertial frame and a standard body frame: x_b forward out the nose, y_b out the right wing, z_b down through the belly. This means *positive p_D is downward*, so altitude is $h = -p_D$. Write that on the first line and never forget it.

2. Quaternion convention. Use scalar-first, $\mathbf{q} = [q_0, q_1, q_2, q_3]^T$, representing rotation from body to NED, with unit norm:

$$q_0^2 + q_1^2 + q_2^2 + q_3^2 = 1. \quad (1)$$

The rotation matrix from body to NED is

$$\mathbf{R}_b^n = \begin{bmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1q_2 - q_0q_3) & 2(q_1q_3 + q_0q_2) \\ 2(q_1q_2 + q_0q_3) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2q_3 - q_0q_1) \\ 2(q_1q_3 - q_0q_2) & 2(q_2q_3 + q_0q_1) & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{bmatrix}. \quad (2)$$

3. State vector. Thirteen states:

$$\mathbf{x} = [p_N, p_E, p_D, u, v, w, q_0, q_1, q_2, q_3, p, q, r]^T \in \mathbb{R}^{13}. \quad (3)$$

4. Control sign convention. Pick one and write it down. A common one: positive elevator $\delta_e > 0$ produces a *nose-down* pitching moment; positive aileron produces a *right-roll* moment; positive rudder produces a *nose-right* yawing moment. Whichever you choose, every derivative sign in phase 2 depends on it.

5. Aircraft parameters. Fill in this table with real numbers before writing a line of code. Values below are a plausible small agile UAV — replace with your own and cite the source.

Mass m	13.5 kg	Wing area S	0.55 m ²
Span b	2.90 m	Mean chord $\bar{c}$	0.19 m
I_{xx}	0.824 kg m ²	I_{yy}	1.135 kg m ²
I_{zz}	1.759 kg m ²	I_{xz}	0.120 kg m ²
Cruise V	25 m/s	Envelope	15–40 m/s, 0–1000 m

0.2 What you code

Nothing executable — but you do create the repository and the configuration file that every later phase reads from.

```
uav_sim/
  config/aircraft_v1.yaml # frozen parameters, version controlled
  docs/conventions.md # the document you just wrote
  src/uav_sim/ # (empty for now)
  tests/ # (empty for now)
```

```
# config/aircraft_v1.yaml
name: agile_uav_v1
frames: {inertial: NED, body: FRD, quaternion: scalar_first}
mass: {m: 13.5, Ixx: 0.824, Iyy: 1.135, Izz: 1.759, Ixz: 0.120}
geometry: {S: 0.55, b: 2.90, c: 0.19}
limits:
  elevator: {min: -0.436, max: 0.436, rate: 5.0, tau: 0.05} # rad, rad/s, s
  aileron: {min: -0.436, max: 0.436, rate: 5.0, tau: 0.05}
  rudder: {min: -0.436, max: 0.436, rate: 5.0, tau: 0.05}
  throttle: {min: 0.0, max: 1.0, rate: 2.0, tau: 0.20}
integration: {method: rk4, dt: 0.002}
```

GATE — do not start the next phase until:

- docs/conventions.md exists and defines all five items above.
- The parameter table has real numbers, with a source noted for each.
- Git repository initialised, first commit made.
- You can answer, without looking: “if $p_D = -100$, what is the altitude?”

1 Phase 1 — Make it move (6-DOF rigid body)

In plain words. This phase teaches the computer that a plane is a heavy object floating in space. Nothing about wings yet — just: if I push it here, how does it move and spin?

The computer holds a list of thirteen numbers describing the plane right now. You write one function that takes that list and says “here is how fast each of those numbers is changing”. Then a second function nudges all thirteen numbers forward by two thousandths of a second. Run it in a loop and the plane moves.

To test it, you turn off the wings and the engine and just drop the plane. If it falls like a stone, in a perfect parabola, your maths is right. If it drifts sideways or gains energy from nowhere, you have a bug — and you must fix it now, because everything else is built on top.

1.1 The maths

Four coupled equation blocks. Newton for translation, Euler for rotation, plus kinematics for position and attitude.

Position kinematics — body velocity rotated into NED:

$$\dot{\mathbf{p}}^n = \mathbf{R}_b^n \mathbf{v}^b. \quad (4)$$

Translational dynamics — note the Coriolis term $\boldsymbol{\omega} \times \mathbf{v}$, which appears because the body frame is rotating:

$$\dot{\mathbf{v}}^b = \frac{1}{m} \mathbf{F}^b - \boldsymbol{\omega}^b \times \mathbf{v}^b, \quad \text{i.e.} \quad \begin{aligned} \dot{u} &= \frac{F_x}{m} - qw + rv \\ \dot{v} &= \frac{F_y}{m} - ru + pw \\ \dot{w} &= \frac{F_z}{m} - pv + qu \end{aligned} \quad (5)$$

Attitude kinematics — quaternion rate:

$$\dot{\mathbf{q}} = \frac{1}{2} \begin{bmatrix} 0 & -p & -q & -r \\ p & 0 & r & -q \\ q & -r & 0 & p \\ r & q & -p & 0 \end{bmatrix} \mathbf{q} \quad (6)$$

Rotational dynamics:

$$\dot{\boldsymbol{\omega}}^b = \mathbf{I}^{-1} \left(\mathbf{M}^b - \boldsymbol{\omega}^b \times (\mathbf{I} \boldsymbol{\omega}^b) \right) \quad (7)$$

with

$$\mathbf{I} = \begin{bmatrix} I_{xx} & 0 & -I_{xz} \\ 0 & I_{yy} & 0 \\ -I_{xz} & 0 & I_{zz} \end{bmatrix}. \quad (8)$$

Gravity in the body frame (the only force in this phase):

$$\mathbf{F}_{\text{grav}}^b = m (\mathbf{R}_b^n)^T \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix}, \quad g = 9.80665 \text{ m/s}^2. \quad (9)$$

Integration. Classical fourth-order Runge–Kutta with step h :

$$\begin{aligned} \mathbf{k}_1 &= f(t, \mathbf{x}), & \mathbf{k}_2 &= f\left(t + \frac{h}{2}, \mathbf{x} + \frac{h}{2} \mathbf{k}_1\right), \\ \mathbf{k}_3 &= f\left(t + \frac{h}{2}, \mathbf{x} + \frac{h}{2} \mathbf{k}_2\right), & \mathbf{k}_4 &= f(t + h, \mathbf{x} + h \mathbf{k}_3), \end{aligned} \quad (10)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{h}{6} (\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4). \quad (11)$$

After each full step, renormalise the quaternion: $\mathbf{q} \leftarrow \mathbf{q} / \|\mathbf{q}\|$.

1.2 What you code

```
# src/uav_sim/dynamics.py

def quat_to_dcm(q: np.ndarray) -> np.ndarray:
    """Body-to-NED rotation matrix. Input shape (4,), output (3,3)."""

def quat_derivative(q: np.ndarray, omega: np.ndarray) -> np.ndarray:
    """Quaternion rate given body angular rates. Output shape (4,)."""

def rigid_body_derivatives(x: np.ndarray,
                           forces_body: np.ndarray,
                           moments_body: np.ndarray,
                           params: AircraftParams) -> np.ndarray:
    """
    Full 13-state derivative. Forces/moments are TOTALS in the body frame,
    including gravity. This function knows nothing about aerodynamics.
    Returns xdot of shape (13,).
    """

# src/uav_sim/integrate.py

def rk4_step(f, t: float, x: np.ndarray, h: float, *args) -> np.ndarray:
    """One RK4 step. Renormalises quaternion elements 6:10 on exit."""

def simulate(x0, controller, duration, dt, params) -> SimResult:
    """Main loop. Returns time histories in a structured result object."""
```

1.3 The gate tests, in detail

1. **Free fall.** Zero forces except gravity, level attitude, zero initial velocity. After t seconds the plane must have fallen exactly $\frac{1}{2}gt^2$ metres, to within 10^{-9} m. This catches gravity sign errors and integration bugs.
2. **Ballistic.** Launch at 45° with speed V_0 . The trajectory must match the analytic parabola, and the range must be $V_0^2 \sin(2 \cdot 45^\circ)/g$.
3. **Quaternion norm.** Over a 100-second simulation with a constant non-zero ω , the norm must stay within 10^{-10} of 1.
4. **Angular momentum.** With zero moments and an asymmetric inertia tensor, the plane must tumble (this is real physics — the intermediate axis theorem), but the total angular momentum $\|\mathbf{R}_b^n \mathbf{I} \omega\|$ in the inertial frame must be constant to within 10^{-8} .
5. **Energy.** With zero forces and moments, total kinetic energy $\frac{1}{2}m\|\mathbf{v}\|^2 + \frac{1}{2}\omega^T \mathbf{I} \omega$ must be constant.

GATE — do not start the next phase until:

All five tests pass in `pytest`, automatically, with numeric tolerances written into the test file. If test 4 fails, your $\omega \times (\mathbf{I}\omega)$ term is wrong or missing — this is the single most common bug in the whole project.

2 Phase 2 — Give it wings (aerodynamics and propulsion)

In plain words. Now the plane gets wings. The idea is simple: the plane is moving through air, and the air pushes back. How hard it pushes depends on how fast you are going and at what angle you are hitting the air.

The clever trick engineers use is to split the push into two parts: a part that depends on speed (easy, always the same formula) and a part that depends on the *shape and angle* of the plane (a number between roughly -2 and 2 called a coefficient). You look up or estimate the coefficients once, and then the formula works at every speed.

The most important number is the angle of attack — how much the nose is tilted up compared to where the plane is actually travelling. Tilt up a little, more lift. Tilt up too much, and the wing stalls and stops working.

2.1 The maths

Air-relative quantities from the body velocity:

$$V = \sqrt{u^2 + v^2 + w^2}, \quad \alpha = \arctan\left(\frac{w}{u}\right), \quad \beta = \arcsin\left(\frac{v}{V}\right). \quad (12)$$

Dynamic pressure:

$$\bar{q} = \frac{1}{2}\rho V^2. \quad (13)$$

Force coefficients — linear build-up first, tables later:

$$C_L = C_{L_0} + C_{L_\alpha}\alpha + C_{L_q}\frac{q\bar{c}}{2V} + C_{L_{\delta_e}}\delta_e \quad (14)$$

$$C_D = C_{D_0} + \frac{C_L^2}{\pi e AR} \quad (\text{with } AR = b^2/S) \quad (15)$$

$$C_Y = C_{Y_\beta}\beta + C_{Y_p}\frac{pb}{2V} + C_{Y_r}\frac{rb}{2V} + C_{Y_{\delta_a}}\delta_a + C_{Y_{\delta_r}}\delta_r \quad (16)$$

Moment coefficients:

$$C_l = C_{l_\beta}\beta + C_{l_p}\frac{pb}{2V} + C_{l_r}\frac{rb}{2V} + C_{l_{\delta_a}}\delta_a + C_{l_{\delta_r}}\delta_r \quad (17)$$

$$C_m = C_{m_0} + C_{m_\alpha}\alpha + C_{m_q}\frac{q\bar{c}}{2V} + C_{m_{\delta_e}}\delta_e \quad (18)$$

$$C_n = C_{n_\beta}\beta + C_{n_p}\frac{pb}{2V} + C_{n_r}\frac{rb}{2V} + C_{n_{\delta_a}}\delta_a + C_{n_{\delta_r}}\delta_r \quad (19)$$

Wind to body axes. Lift and drag act in the wind frame, so rotate them:

$$\begin{bmatrix} F_x \\ F_z \end{bmatrix}_{\text{aero}} = \bar{q}S \begin{bmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{bmatrix} \begin{bmatrix} -C_D \\ -C_L \end{bmatrix}, \quad F_y = \bar{q}S C_Y. \quad (20)$$

Moments:

$$\mathbf{M}_{\text{aero}}^b = \bar{q}S \begin{bmatrix} b C_l \\ \bar{c} C_m \\ b C_n \end{bmatrix}. \quad (21)$$

Propulsion — a simple propeller momentum model:

$$F_{x,\text{prop}} = \frac{1}{2}\rho S_{\text{prop}} C_{\text{prop}} [(k_{\text{motor}}\delta_t)^2 - V^2]. \quad (22)$$

Atmosphere — ISA troposphere, needed because density falls with altitude and that changes everything:

$$T = T_0 - Lh, \quad \rho = \rho_0 \left(\frac{T}{T_0} \right)^{\frac{g}{LR}-1}, \quad (23)$$

with $T_0 = 288.15$ K, $L = 0.0065$ K/m, $R = 287.053$ J/(kg K).

2.2 What you code

```
# src/uav_sim/aero.py

class AeroModel(Protocol):
    """The swap point. Linear model now; lookup tables or a GP later
    can implement this same interface without touching dynamics.py."""
    def coefficients(self, alpha, beta, p, q, r, V, controls) -> AeroCoeffs: ...

class LinearAeroModel(AeroModel):
    def __init__(self, derivs: dict): ...
    def coefficients(self, ...) -> AeroCoeffs: ...

def aero_forces_moments(x, controls, rho, params, model: AeroModel):
    """Returns (F_body, M_body) as (3,) arrays. Pure function, no state."""

# src/uav_sim/propulsion.py
def thrust(delta_t: float, V: float, rho: float, params) -> np.ndarray: ...

# src/uav_sim/atmosphere.py
def isa(altitude_m: float) -> Atmosphere:
    """Returns rho, temperature, pressure, speed of sound."""
```

2.3 The gate tests

1. **Monotonic lift.** C_L must increase with α over the linear range, and lift force must be positive upward (negative F_z in body axes) at positive α .
2. **Static stability sign.** $C_{m_\alpha} < 0$. If it is positive your aircraft is statically unstable and will diverge in pitch — almost certainly a sign error, not a design choice.
3. **Damping signs.** $C_{m_q} < 0$, $C_{l_p} < 0$, $C_{n_r} < 0$. All three oppose motion. A positive damping derivative means your simulator generates energy from nothing.
4. **Symmetry.** With $\beta = 0$, $\delta_a = 0$, $\delta_r = 0$, $p = r = 0$, the lateral force and both lateral moments must be *exactly* zero.
5. **Density.** $\rho(0) = 1.225 \text{ kg/m}^3$ to 4 significant figures, and ρ decreases monotonically with altitude.

Trap. Do not proceed if any sign test fails. A wrong sign here produces an aircraft that flies “fine” in some conditions and diverges inexplicably in others, and you will spend weeks blaming your controller.

GATE — do not start the next phase until:

All five tests pass. Additionally, a hand-flown open-loop simulation (constant small elevator, fixed throttle) produces a plausible-looking trajectory: the aircraft should pitch, oscillate, and not immediately explode to infinity.

3 Phase 3 — Find level flight and the linear model

In plain words. A plane in steady cruise is perfectly balanced: the lift exactly cancels the weight, the thrust exactly cancels the drag, and nothing is turning. There is one exact combination of nose angle and throttle that does this. Finding it is called “trimming”.

You do not solve it with algebra — you let the computer hunt. It guesses a nose angle and throttle, checks how unbalanced the plane still is, and adjusts, over and over, until the imbalance is essentially zero.

Once balanced, you do something sneaky. You poke the plane very gently in every possible direction and record how it responds. Those responses form a simple table of numbers that behaves almost exactly like the real plane, as long as you stay near the balanced point. Simple tables are easy to design controllers for. And crucially, this table tells you the plane’s natural wobbles — and you can check those wobbles against what real aircraft actually do. This is your first proof that the whole simulator is correct.

3.1 The maths

Trim as a root-finding problem. You want a state and input such that the aircraft is in steady flight at airspeed V^* and flight path angle γ^* . Define the residual

$$\mathbf{g}(\alpha, \theta, \delta_e, \delta_t) = \begin{bmatrix} \dot{u} \\ \dot{w} \\ \dot{q} \\ \gamma - \gamma^* \end{bmatrix}, \quad (24)$$

and solve $\mathbf{g} = \mathbf{0}$ numerically. Equivalently, minimise the scalar cost

$$J_{\text{trim}} = \|\dot{\mathbf{x}}_{\text{trim states}}\|_2^2 \quad (25)$$

using `scipy.optimize.fsolve` or `minimize`. Convergence means $J_{\text{trim}} < 10^{-10}$.

Linearisation by central differences. With the trim point $(\mathbf{x}^*, \mathbf{u}^*)$ found, build the Jacobians numerically:

$$A_{ij} = \frac{\partial f_i}{\partial x_j} \approx \frac{f_i(\mathbf{x}^* + \epsilon \mathbf{e}_j, \mathbf{u}^*) - f_i(\mathbf{x}^* - \epsilon \mathbf{e}_j, \mathbf{u}^*)}{2\epsilon}, \quad (26)$$

$$B_{ij} = \frac{\partial f_i}{\partial u_j} \approx \frac{f_i(\mathbf{x}^*, \mathbf{u}^* + \epsilon \mathbf{e}_j) - f_i(\mathbf{x}^*, \mathbf{u}^* - \epsilon \mathbf{e}_j)}{2\epsilon}, \quad (27)$$

with $\epsilon \approx 10^{-6}$. The result is the familiar local model

$$\delta \dot{\mathbf{x}} = A \delta \mathbf{x} + B \delta \mathbf{u}. \quad (28)$$

Modes. Compute $\lambda_i = \text{eig}(A)$. Each complex pair gives a natural frequency and damping ratio:

$$\omega_n = |\lambda|, \quad \zeta = -\frac{\text{Re}(\lambda)}{|\lambda|}. \quad (29)$$

A correct fixed-wing aircraft *must* show these modes:

Mode	Axis	Typical ω_n	Typical ζ
Short period	Longitudinal	3–10 rad/s	0.3–0.9
Phugoid	Longitudinal	0.1–0.5 rad/s	0.02–0.2
Dutch roll	Lateral	1–5 rad/s	0.05–0.4
Roll subsidence	Lateral	real, $\tau \approx 0.05$ –0.5 s	—
Spiral	Lateral	real, slow, may be unstable	—

3.2 What you code

```
# src/uav_sim/trim.py

def trim(V_star: float, gamma_star: float, altitude: float,
         params, model) -> TrimPoint:
    """
    Solves for (x_star, u_star) in steady flight.
    Raises TrimNotConverged if residual > 1e-10.
    """

# src/uav_sim/linearise.py

def linearise(x_star, u_star, params, model, eps=1e-6) -> tuple[np.ndarray, np.ndarray]:
    """Returns (A, B) by central differences on the nonlinear derivative fn."""

def modes(A: np.ndarray) -> list[Mode]:
    """Eigen-decomposition -> named modes with omega_n and zeta."""
```

3.3 The gate tests

1. **Trim converges** at 6 points across the envelope ($V \in \{18, 25, 35\}$ m/s $\times \gamma \in \{-5^\circ, 0^\circ, +5^\circ\}$), each with residual below 10^{-10} .
2. **Trim is physically sensible.** As V decreases, α must increase. As γ increases (climb), δ_t must increase.
3. **All five modes present**, with frequencies and damping inside the ranges in the table above.
4. **Linear matches nonlinear.** Apply a small elevator doublet (0.5°) to both the nonlinear simulator and the linear model. Over 5 seconds the state trajectories must agree to within 2%. Then apply a large one (15°) and confirm they *diverge* — this proves your linearisation is genuinely local, and justifies the nonlinear controller in phase 6.

GATE — do not start the next phase until:

Trim converges everywhere in the envelope, all five modes appear with plausible values, and the linear/nonlinear small-signal comparison agrees. This is the most important gate in the project: passing it is your evidence that the model is right.

4 Phase 4 — First autopilot (cascaded PID)

In plain words. Now you write the pilot. Not a clever one — a simple one, the kind that just says “I want to be pointing 10 degrees up; I am pointing 8; that is 2 too few; push the stick a bit”.

The trick is to stack three of these on top of each other, like nested dolls. The innermost one is very fast and controls how quickly the plane is rolling. The middle one is slower and controls the angle the plane is banked at. The outer one is slowest and controls where the plane is going. Each layer just tells the layer inside it what to aim for.

Why start with the dumb pilot? Because later, when you build a clever pilot, you need something to beat. If you cannot say “the clever one is 40% better than the simple one”, you have not proved anything at all.

4.1 The maths

The PID law in continuous form:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}, \quad e(t) = r(t) - y(t). \quad (30)$$

Discrete implementation at sample time T_s , using a derivative on *measurement* (not error) to avoid derivative kick, and a filter constant N :

$$u_k = K_p e_k + I_k + D_k, \quad (31)$$

$$I_k = I_{k-1} + K_i T_s e_k, \quad D_k = \frac{K_d}{K_d + N T_s} D_{k-1} - \frac{K_d N}{K_d + N T_s} (y_k - y_{k-1}). \quad (32)$$

Anti-windup (essential once actuators saturate in phase 5) — back-calculation:

$$I_k = I_{k-1} + K_i T_s e_k + \frac{T_s}{T_t} (\text{sat}(u_k) - u_k). \quad (33)$$

The cascade. Three loops, with a strict frequency separation rule: each outer loop must be at least 3–5 times slower than the loop inside it, otherwise they fight each other.

Loop	Input	Output	Rate
Inner (rate)	p, q, r commands	$\delta_a, \delta_e, \delta_r$	200–500 Hz
Middle (attitude)	ϕ, θ commands	p, q commands	50–100 Hz
Outer (guidance)	altitude, airspeed, path	ϕ, θ, δ_t commands	10–20 Hz

Total energy control for the outer longitudinal loop — elevator controls the *balance* of energy, throttle controls the *total*:

$$E_{\text{total}} = \frac{1}{2} m V^2 + mgh, \quad E_{\text{balance}} = \frac{1}{2} m V^2 - mgh. \quad (34)$$

4.2 What you code

```
# src/uav_sim/control/pid.py

class PID:
    def __init__(self, kp, ki, kd, N=20.0, limits=(-inf, inf), Tt=None): ...
    def update(self, setpoint: float, measurement: float, dt: float) -> float:
        """Includes derivative-on-measurement and back-calculation anti-windup."""
    def reset(self): ...
```

```
# src/uav_sim/control/baseline.py

class CascadedPIDAutopilot(Controller):
    """Controller protocol: .update(state_estimate, reference, t) -> controls"""
    def __init__(self, gains: GainSet): ...
    def update(self, x_hat, ref: Reference, t: float) -> Controls: ...
```

4.3 The gate tests

1. **Step response, perfect world.** With no wind, no noise, ideal actuators: a 10° roll command settles within 2 s, overshoot below 20%, steady-state error below 0.5° .
2. **Altitude hold.** A 50 m step altitude command is captured without airspeed dropping more than 3 m/s.
3. **Frequency separation verified** by measuring the closed-loop bandwidth of each loop and confirming the $3\text{--}5\times$ ratios.
4. **Envelope coverage.** The same gain set works at $V = 18$ and $V = 35$ m/s. If it does not, you have found your motivation for gain scheduling — note this down, it belongs in the report.

GATE — do not start the next phase until:

The baseline flies a full test mission (climb, turn, level, descend) in the perfect world, and the performance numbers are *recorded to a file*. These numbers are the reference every later phase is measured against.

5 Phase 5 — Make the world unfair

In plain words. Everything so far has been a fantasy. The plane knew exactly where it was, its flaps moved instantly, and the air was perfectly still. None of that is true.

So now you break it, deliberately, in four ways. The eyes get blurry: the sensors report the truth plus a wobble plus a slowly drifting error. The muscles get slow: the flaps take time to move, cannot move faster than a certain speed, and cannot go past a certain angle. The air starts pushing: steady wind plus random gusts. And the plane no longer knows exactly where it is, so it has to guess from the blurry sensors.

Then you run your phase-4 autopilot again and watch it get worse. That is not a failure — that is your first real result. Write the number down. The whole rest of the project is about closing that gap.

5.1 The maths

Actuators — first-order lag, then rate limit, then saturation, in that order:

$$\dot{\delta} = \frac{1}{\tau} (\delta_{\text{cmd}} - \delta), \quad \dot{\delta} \leftarrow \text{clip}(\dot{\delta}, -\dot{\delta}_{\text{max}}, +\dot{\delta}_{\text{max}}), \quad \delta \leftarrow \text{clip}(\delta, \delta_{\text{min}}, \delta_{\text{max}}). \quad (35)$$

Sensors — measurement equals truth plus slowly drifting bias plus white noise:

$$y_k = x_k + b_k + n_k, \quad n_k \sim \mathcal{N}(0, \sigma^2), \quad b_{k+1} = b_k + w_k, \quad w_k \sim \mathcal{N}(0, \sigma_b^2 T_s). \quad (36)$$

Typical MEMS values to start from:

Sensor	Rate	Noise σ	Bias drift
Gyroscope	200 Hz	0.01 rad/s	0.001 rad/s/ $\sqrt{s}$
Accelerometer	200 Hz	0.05 m/s ²	0.005 m/s ² / $\sqrt{s}$
GPS position	5 Hz	2.0 m	—
Pitot (airspeed)	50 Hz	0.5 m/s	0.1 m/s
Barometer	50 Hz	0.5 m	0.2 m

Wind and turbulence. Total airspeed vector becomes $\mathbf{v}_{\text{air}} = \mathbf{v}^b - (\mathbf{R}_b^n)^T \mathbf{v}_{\text{wind}}^n$, and *every* aerodynamic calculation must use $\mathbf{v}_{\text{air}}$, not $\mathbf{v}^b$. Turbulence uses the Dryden spectrum; the longitudinal component is

$$\Phi_u(\Omega) = \sigma_u^2 \frac{2L_u}{\pi} \frac{1}{1 + (L_u \Omega)^2}, \quad (37)$$

implemented as a shaping filter driven by white noise:

$$H_u(s) = \sigma_u \sqrt{\frac{2L_u}{\pi V}} \frac{1}{1 + \frac{L_u}{V}s}. \quad (38)$$

State estimation — Extended Kalman Filter. Predict:

$$\hat{\mathbf{x}}_k^- = f(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_{k-1}), \quad P_k^- = F_k P_{k-1} F_k^T + Q. \quad (39)$$

Update:

$$K_k = P_k^- H_k^T (H_k P_k^- H_k^T + R)^{-1}, \quad (40)$$

$$\hat{\mathbf{x}}_k = \hat{\mathbf{x}}_k^- + K_k (\mathbf{y}_k - h(\hat{\mathbf{x}}_k^-)), \quad P_k = (I - K_k H_k) P_k^-. \quad (41)$$

5.2 What you code

```
# src/uav_sim/hardware/actuator.py
class Actuator:
    def __init__(self, tau, rate_limit, pos_limits): ...
    def step(self, command: float, dt: float) -> float:
        """Order matters: lag -> rate limit -> saturation."""

# src/uav_sim/hardware/sensors.py
class SensorSuite:
    def __init__(self, spec: SensorSpec, rng: np.random.Generator): ...
    def measure(self, x_true, t) -> Measurements:
        """Respects each sensor's own update rate; holds between samples."""

# src/uav_sim/environment/wind.py
class DrydenTurbulence:
    def __init__(self, sigma, length_scale, rng): ...
    def step(self, V: float, dt: float) -> np.ndarray: ...

# src/uav_sim/estimation/ekf.py
class AttitudeEKF:
    def predict(self, imu, dt): ...
    def update_gps(self, gps): ...
    def update_baro(self, baro): ...
```

5.3 The gate tests

1. **Rate limit works.** Command a step to full deflection; the surface must take exactly $\Delta\delta/\dot{\delta}_{\max}$ seconds to arrive.
2. **Noise statistics.** Over 100,000 samples, the measured noise standard deviation matches the spec to within 5%.
3. **Reproducibility.** Two runs with the same seed produce bit-identical outputs. Two runs with different seeds do not. This is non-negotiable for phase 9.
4. **EKF converges.** Estimated attitude error stays below 2° RMS in steady flight, and the filter recovers from a 30° initial error within 10 seconds.
5. **Degradation is measured.** Re-run the phase-4 mission with everything enabled and record the new numbers next to the old ones.

GATE — do not start the next phase until:

You have a table with two columns — perfect world and realistic world — showing exactly how much the baseline controller lost. This table is the answer to research question RQ1 and belongs in the report.

6 Phase 6 — The better autopilot

In plain words. The simple pilot from phase 4 works by noticing mistakes and correcting them. It has no idea *why* the plane moves — it just reacts. That is fine when things are gentle, but it struggles when the plane behaves very differently at 18 m/s than at 35 m/s.

The clever pilot is different. It has a copy of the physics inside it. So instead of reacting, it calculates: “I want the nose to rotate at exactly this rate. Physics says the plane is currently producing that much moment by itself. So I need the elevator to supply precisely the difference.” It cancels out the aircraft’s own behaviour and replaces it with the behaviour you asked for.

The catch — and this is the whole research question — is that its internal copy of the physics is never exactly right. So how wrong can it be before the clever pilot becomes worse than the simple one?

6.1 The maths: nonlinear dynamic inversion

Write the rotational dynamics in control-affine form:

$$\dot{\boldsymbol{\omega}} = \underbrace{\mathbf{I}^{-1} (\mathbf{M}_0(\mathbf{x}) - \boldsymbol{\omega} \times \mathbf{I}\boldsymbol{\omega})}_{f(\mathbf{x})} + \underbrace{\mathbf{I}^{-1} \bar{q} S \mathbf{G}}_{g(\mathbf{x})} \boldsymbol{\delta}, \quad (42)$$

where $\mathbf{M}_0$ collects all control-independent moments and $\mathbf{G}$ is the control effectiveness matrix

$$\mathbf{G} = \begin{bmatrix} b C_{l_{\delta_a}} & 0 & b C_{l_{\delta_r}} \\ 0 & \bar{c} C_{m_{\delta_e}} & 0 \\ b C_{n_{\delta_a}} & 0 & b C_{n_{\delta_r}} \end{bmatrix}. \quad (43)$$

Choose the *desired* closed-loop dynamics (the virtual control) as a simple first-order response to the rate command:

$$\boldsymbol{\nu} = \mathbf{K}_{\omega} (\boldsymbol{\omega}_{\text{cmd}} - \boldsymbol{\omega}). \quad (44)$$

Then invert:

$$\boxed{\boldsymbol{\delta} = g(\mathbf{x})^{-1} (\boldsymbol{\nu} - f(\mathbf{x}))} \quad (45)$$

If the model were perfect, the closed loop would be exactly $\dot{\boldsymbol{\omega}} = \mathbf{K}_{\omega} (\boldsymbol{\omega}_{\text{cmd}} - \boldsymbol{\omega})$ — a clean, linear, speed-independent response at every point in the envelope.

Robustness patch. Because the model *is* wrong, add an integral term outside the inversion so that steady modelling error is rejected:

$$\boldsymbol{\nu} = \mathbf{K}_{\omega} \mathbf{e}_{\omega} + \mathbf{K}_I \int \mathbf{e}_{\omega} dt, \quad \mathbf{e}_{\omega} = \boldsymbol{\omega}_{\text{cmd}} - \hat{\boldsymbol{\omega}}. \quad (46)$$

Control allocation when $\mathbf{G}$ is not square (more surfaces than axes), use the weighted pseudo-inverse:

$$\boldsymbol{\delta} = W^{-1} \mathbf{G}^T (\mathbf{G} W^{-1} \mathbf{G}^T)^{-1} \boldsymbol{\nu}_{\text{eff}}. \quad (47)$$

Trap. $g(\mathbf{x})^{-1}$ becomes singular at low airspeed, because $\bar{q} \rightarrow 0$ means the surfaces do nothing. Guard it: clamp $\bar{q}$ to a minimum, monitor the condition number of $\mathbf{G}$, and blend back to the PID baseline below a threshold airspeed. An ungarded inversion will produce infinite commands and your simulation will fill with NaNs.

6.2 What you code

```
# src/uav_sim/control/ndi.py

class NDIController(Controller):
    def __init__(self, model: AeroModel, params, K_omega, K_I,
                 q_bar_min=5.0, cond_max=1e4): ...

    def _virtual_control(self, omega_cmd, omega_hat, dt) -> np.ndarray: ...
    def _invert(self, nu, x_hat) -> np.ndarray:
        """Raises SingularInversion if cond(G) > cond_max."""
    def update(self, x_hat, ref, t) -> Controls: ...

# src/uav_sim/control/allocation.py
def weighted_pseudo_inverse(G, nu, W=None, limits=None) -> np.ndarray: ...
```

6.3 Fair comparison rules

This is where most projects quietly cheat. Enforce all four:

1. Both controllers use the *same* state estimate from the *same* EKF.
2. Both are subject to the *same* actuator and sensor models.
3. Both are evaluated on the *same* scenario list with the *same* random seeds.
4. Both are tuned with *comparable effort* — if you spend two weeks on NDI and twenty minutes on the PID, your comparison is meaningless. Tune both with the optimiser in phase 7.

GATE — do not start the next phase until:

NDI flies the same test mission as the baseline, produces the same or better tracking, and degrades *gracefully* when you deliberately corrupt its internal model by $\pm 30\%$ on $C_{m\alpha}$. Record the degradation curve — it is a headline figure in the report.

7 Phase 7 — Auto-tuning (optimisation)

In plain words. Up to now you have been turning the knobs on your autopilot by hand: try a number, watch the plot, try again. That is slow and you will never find the best setting.

So you let the computer do it. You write down, as one single number, what “good flying” means — combining how closely it followed the path, how much it wobbled, how hard it worked the flaps, and how often it hit the limits. Then the computer tries thousands of knob settings and keeps the best.

There is one enormous trap. If you let the computer tune against just a few test flights, it will find settings that are brilliant on *those exact flights* and terrible everywhere else — like a student who memorises last year’s exam paper. So you split your flights into two groups: one to tune with, one to test with. The tuner never gets to see the test group.

7.1 The maths

The objective function. Every term must be normalised by a reference value, otherwise the units dominate the weights:

$$J(\mathbf{k}) = \frac{1}{N_s} \sum_{i=1}^{N_s} \left[w_1 \tilde{E}_{\text{track}}^{(i)} + w_2 \tilde{E}_{\text{rate}}^{(i)} + w_3 \tilde{U}_{\text{ctrl}}^{(i)} + w_4 P_{\text{sat}}^{(i)} + w_5 P_{\text{viol}}^{(i)} \right] \quad (48)$$

with

$$E_{\text{track}} = \int_0^T \|\mathbf{r}(t) - \mathbf{y}(t)\|^2 dt \quad (\text{tracking error}) \quad (49)$$

$$E_{\text{rate}} = \int_0^T \|\dot{\boldsymbol{\delta}}(t)\|^2 dt \quad (\text{actuator wear / smoothness}) \quad (50)$$

$$U_{\text{ctrl}} = \int_0^T \boldsymbol{\delta}(t)^T R \boldsymbol{\delta}(t) dt \quad (\text{control effort}) \quad (51)$$

$$P_{\text{sat}} = \frac{1}{T} \int_0^T \mathbb{1}_{\{|\delta| \geq \delta_{\text{max}}\}} dt \quad (\text{fraction of time saturated}) \quad (52)$$

$$P_{\text{viol}} = \sum_j \max \left(0, \frac{c_j - c_j^{\text{max}}}{c_j^{\text{max}}} \right)^2 \quad (\text{envelope violations}) \quad (53)$$

and normalisation $\tilde{E} = E/E_{\text{ref}}$ using the baseline controller’s values, so that $J = 1$ means “as good as the baseline”.

Hard failure. If a run diverges, hits NaN, or exceeds the flight envelope, return $J = 10^6$ rather than a NaN — the optimiser must be able to compare it.

Optimiser choice.

Method	Use when	Cost
Nelder–Mead (SciPy)	< 6 gains, quick first pass	200–500 evals
CMA-ES	6–30 gains, noisy J , no gradients	1000–5000 evals
Bayesian optimisation	each eval is very expensive	50–200 evals

CMA-ES is the recommended primary choice: it handles the noisy, non-smooth, non-convex objective a flight simulator produces.

The train/validation split. Generate N scenarios, partition disjointly:

$$\mathcal{S} = \mathcal{S}_{\text{train}} \sqcup \mathcal{S}_{\text{val}}, \quad |\mathcal{S}_{\text{train}}| = 40, \quad |\mathcal{S}_{\text{val}}| = 200. \quad (54)$$

Overfitting is quantified by the generalisation gap:

$$\Delta = \frac{J_{\text{val}}(\mathbf{k}^*) - J_{\text{train}}(\mathbf{k}^*)}{J_{\text{train}}(\mathbf{k}^*)}. \quad (55)$$

If $\Delta > 0.2$, you have overfitted. Fix it by enlarging $\mathcal{S}_{\text{train}}$ or by adding a regulariser $\lambda\|\mathbf{k}\|^2$.

7.2 What you code

```
# src/uav_sim/optimise/objective.py

def evaluate_gains(k: np.ndarray, scenarios: list[Scenario],
                  controller_factory, params) -> float:
    """
    Runs every scenario, returns scalar J. Deterministic given the
    scenario seeds. Returns 1e6 on divergence rather than NaN.
    Parallelised with multiprocessing.Pool.
    """

# src/uav_sim/optimise/run_cmaes.py

def tune(controller_factory, k0, bounds, train_set, val_set,
        sigma0=0.3, budget=2000) -> TuningResult:
    """Returns best gains, J_train, J_val, full convergence history."""
```

7.3 The gate tests

1. **Objective is deterministic.** Calling `evaluate_gains` twice with identical input returns identical output, to the last bit.
2. **Optimiser converges.** J decreases monotonically in the best-so-far sense, and flattens before the budget is exhausted.
3. **No overfitting.** $\Delta < 0.2$ on the held-out validation set.
4. **Both controllers tuned.** The PID baseline and the NDI controller have each been through the same optimiser with the same budget. Otherwise your phase-6 comparison is invalid and you must redo it.

GATE — do not start the next phase until:

Both controllers have optimised gains, the generalisation gap is small, and the tuned NDI still beats the tuned PID. If it no longer does, that is a legitimate and interesting result — report it honestly rather than hiding it.

8 Phase 8 — Learn the leftovers (optional but impressive)

In plain words. Your physics model is a simplification. The real plane does slightly different things — especially at big angles, where air does complicated swirly stuff that simple equations do not capture.

So here is a nice idea: do not replace the physics with a neural network. Keep the physics, and train a small model to predict only the *difference* between what the physics said and what actually happened. The physics does the heavy lifting; the learned bit fills the gaps.

And you use a special kind of model called a Gaussian Process, because it does something unusual: as well as guessing the answer, it tells you *how confident it is*. Feed it a flight condition it has never seen and it says “I have no idea here”. That lets you draw a map of the flight envelope showing exactly where your model is trustworthy and where it is not. That map is a genuinely publishable result.

8.1 The maths

Physics-informed residual structure:

$$\hat{f}(\mathbf{x}, \mathbf{u}) = \underbrace{f_{\text{phys}}(\mathbf{x}, \mathbf{u})}_{\text{your phase-2 model}} + \underbrace{f_{\text{ML}}(\mathbf{x}, \mathbf{u})}_{\text{learned correction}}. \quad (56)$$

The training target is the residual $\mathbf{r} = \mathbf{y}_{\text{truth}} - f_{\text{phys}}$, where the truth comes from a higher-fidelity source: a nonlinear lookup-table model, published wind-tunnel data, or logged flight data.

Gaussian Process regression. With training inputs X , targets $\mathbf{y}$, kernel $k(\cdot, \cdot)$ and noise σ_n^2 , the prediction at a new point $\mathbf{x}_*$ is Gaussian with

$$\mu(\mathbf{x}_*) = \mathbf{k}_*^T (K + \sigma_n^2 I)^{-1} \mathbf{y}, \quad (57)$$

$$\sigma^2(\mathbf{x}_*) = k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}_*^T (K + \sigma_n^2 I)^{-1} \mathbf{k}_*. \quad (58)$$

A squared-exponential kernel with automatic relevance determination:

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp \left(-\frac{1}{2} \sum_{d=1}^D \frac{(x_d - x'_d)^2}{\ell_d^2} \right), \quad (59)$$

where the learned length-scales ℓ_d additionally tell you *which inputs actually matter* — a free interpretability result.

Calibration check. A well-calibrated GP has residuals whose normalised values are standard normal:

$$z_i = \frac{y_i - \mu(\mathbf{x}_i)}{\sigma(\mathbf{x}_i)} \sim \mathcal{N}(0, 1), \quad (60)$$

so roughly 95% of held-out points must fall inside $\mu \pm 1.96\sigma$.

8.2 What you code

```
# src/uav_sim/models/gp_residual.py

class GPResidualAero(AeroModel):
    """Implements the SAME AeroModel interface as phase 2, so it drops
    straight into the simulator with no changes to dynamics.py."""

    def fit(self, X: np.ndarray, residuals: np.ndarray): ...
    def coefficients(self, alpha, beta, p, q, r, V, controls) -> AeroCoeffs: ...
```

```
def uncertainty(self, alpha, beta, V) -> float:
    """Predictive std dev -- used for the confidence map."""

# scripts/build_confidence_map.py
# Sweeps (alpha, V) grid, evaluates GP sigma, exports a heatmap.
```

8.3 The gate tests

1. **Held-out improvement.** Residual RMSE on unseen data is lower with the GP than without it. If not, the GP is memorising noise — do not use it.
2. **Calibration.** 95% of held-out points lie inside the 95% predictive interval, $\pm 3\%$.
3. **Extrapolation sanity.** Query the GP far outside the training data; σ must grow large. A model that is confidently wrong outside its training set is dangerous and must not go anywhere near a control loop.
4. **Interface unchanged.** `dynamics.py` required zero edits to accept the GP model. If you had to modify it, your phase-2 abstraction was wrong.

GATE — do not start the next phase until:

A confidence map over the (α, V) envelope exists, showing where the model is trustworthy. This directly answers research question RQ4.

9 Phase 9 — Ten thousand flights

In plain words. One test flight proves nothing. Maybe you got lucky. Maybe the wind happened to blow the right way.

So you build ten thousand slightly different planes. One is a bit heavier. One has slightly weaker wings. One flies through nastier gusts. One has a sensor that drifts more than usual. Each one gets a random combination, drawn from ranges you decided in advance based on how uncertain you honestly are.

Then you fly all ten thousand, with both autopilots, and count how many succeeded. At the end you do not have a pretty picture — you have a number with error bars. That is what makes this an engineering study rather than a demo.

9.1 The maths

Sample the uncertainty. Draw $\theta^{(i)} \sim p(\theta)$ for scenario i :

Parameter	Distribution	Type
m	$\mathcal{N}(m_0, (0.05 m_0)^2)$	aleatoric
x_{CG}	$\mathcal{U}(-0.03\bar{c}, +0.03\bar{c})$	aleatoric
I_{xx}, I_{yy}, I_{zz}	$\mathcal{N}(I_0, (0.10 I_0)^2)$	epistemic
$C_{L\alpha}, C_{m\alpha}$	$\mathcal{N}(C_0, (0.15 C_0)^2)$	epistemic
C_{mq}, C_{lp}, C_{nr}	$\mathcal{N}(C_0, (0.20 C_0)^2)$	epistemic
Steady wind speed	$\mathcal{U}(0, 12)$ m/s	aleatoric
Wind direction	$\mathcal{U}(0, 2\pi)$	aleatoric
Turbulence intensity	light / moderate / severe, $p = [0.5, 0.35, 0.15]$	aleatoric
Actuator τ	$\mathcal{U}(0.03, 0.08)$ s	epistemic

Separating *epistemic* uncertainty (things you do not know, reducible with better data) from *aleatoric* uncertainty (things that are genuinely random) is what makes the analysis rigorous rather than decorative.

Define success before you look at any results:

$$S^{(i)} = \mathbb{I}[\text{RMSE}_h < 5 \text{ m} \wedge \text{RMSE}_V < 1.5 \text{ m/s} \wedge \max |\phi| < 60^\circ \wedge \text{no NaN} \wedge P_{\text{sat}} < 0.10] \quad (61)$$

Estimate the success probability:

$$\hat{p} = \frac{1}{N} \sum_{i=1}^N S^{(i)}, \quad (62)$$

with the Wilson score confidence interval (better than the normal approximation when $\hat{p}$ is near 1, which is exactly your situation):

$$\text{CI}_{95\%} = \frac{\hat{p} + \frac{z^2}{2N} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{N} + \frac{z^2}{4N^2}}}{1 + \frac{z^2}{N}}, \quad z = 1.96. \quad (63)$$

Compare the two controllers with McNemar's test on paired outcomes (same seeds, both controllers), where b and c are the disagreement counts:

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c}. \quad (64)$$

This is the correct test because the scenarios are paired — using an unpaired test throws away most of your statistical power.

How many runs? To resolve a success rate near p with half-width w :

$$N \approx \frac{z^2 p(1-p)}{w^2}. \quad (65)$$

For $p = 0.95$ and $w = 0.005$, $N \approx 7300$ — which is where the figure of 10,000 comes from.

9.2 What you code

```
# src/uav_sim/montecarlo/sampling.py
def sample_parameters(rng: np.random.Generator, spec: UncertaintySpec) -> dict: ...

# src/uav_sim/montecarlo/campaign.py
def run_case(seed: int, controller_name: str, spec) -> CaseResult:
    """One fully reproducible run. All randomness derives from 'seed'."""

def run_campaign(n_cases: int, controllers: list[str],
                 out_dir: Path, n_workers: int) -> Path:
    """
    Parallel campaign. Writes one Parquet/JSONL row per case containing:
    seed, controller, git_commit, sampled_params, all metrics, success flag.
    Resumable: skips seeds already present in the output.
    """

# src/uav_sim/analysis/statistics.py
def wilson_interval(k: int, n: int, z=1.96) -> tuple[float, float]: ...
def mcnemar(results_a, results_b) -> TestResult: ...
def convergence_curve(results) -> np.ndarray: ...
```

9.3 The gate tests

1. **Reproducible.** Re-running seed 4271 reproduces the stored result exactly.
2. **Converged.** Plot $\hat{p}$ against N ; it must be flat over the last 2000 cases. If it is still moving, run more.
3. **Provenance.** Every result row records the git commit hash. A result you cannot trace to a code version is not evidence.
4. **Failures inspected.** Take the 10 worst cases and actually look at their time histories. Are they real control failures, or simulator bugs? You will usually find at least one bug this way.

GATE — do not start the next phase until:

You can state, with a confidence interval and a significance test: “Controller B succeeded in $\hat{p}_B$ of cases versus $\hat{p}_A$ for the baseline, $p < 0.001$.” That sentence is the project.

10 Phase 10 — Report and demonstration website

In plain words. Last step: tell people what you found. Not “look at my code” — but “here is a question, here is how I answered it, and here is the proof”.

The website is not a separate project. It reads exactly the same result files that the report’s figures come from. That way they can never disagree with each other, and you are not doing the work twice.

10.1 The export contract

Define this in phase 0 and every phase writes to it. One JSON file per experiment:

```
{
  "experiment_id": "mc_ndi_v3",
  "git_commit": "a3f91c2",
  "config_hash": "8d21ee",
  "controller": "ndi_tuned",
  "n_cases": 10000,
  "success_rate": 0.973,
  "ci95": [0.9696, 0.9761],
  "metrics": {
    "rmse_h": {"median": 1.82, "p95": 4.11, "p99": 5.94, "worst": 9.20},
    "rmse_V": {"median": 0.41, "p95": 0.98, "p99": 1.37, "worst": 2.85}
  },
  "trajectories": ["traj_0001.json", "traj_worst.json", "traj_median.json"]
}
```

10.2 What the website shows, in order

1. **The question**, in one paragraph of plain English.
2. **The aircraft and model** — geometry, coefficient structure, assumptions.
3. **Proof the model is right** — trim convergence, mode table against expected ranges, open-loop response. Most portfolio projects skip this section; it is the one an engineer will read first.
4. **Baseline versus advanced** — both trajectories on one plot, with a wind toggle so the reader watches the gap appear.
5. **The uncertainty** — Monte Carlo histogram, with a slider to scrub individual cases and see the matching trajectory.
6. **The headline number** — success rates with confidence intervals.

Trap. The website must be a *renderer*, not a second simulator. Do not reimplement flight dynamics in JavaScript. Python writes static JSON; the page loads it and draws it. No backend, no server cost, hosts free on GitHub Pages, and it can never disagree with the report. Build it after phase 6, once real results exist — earlier and it becomes the feature creep the project definition warns about.

10.3 Report structure

Follow section 22 of the Project Definition. Two additions worth insisting on:

- **Every figure is generated by a script**, committed to the repo, that reads the results database. No figure is ever assembled by hand.

- **The Limitations section is written honestly and early.** Rigid-body assumption, no aeroelasticity, coefficient model validity range, uncertainty distributions chosen by judgement rather than measured. Stating these clearly makes the rest of the work more credible, not less.

GATE — do not start the next phase until:

Someone who has never seen the project can read the report, run `make reproduce`, and obtain your numbers. That is the definition of finished.

A Appendix A — Suggested repository layout

```

uav_sim/
  config/
    aircraft_v1.yaml # frozen in phase 0
    uncertainty_v1.yaml # distributions for phase 9
    scenarios/train.yaml, val.yaml
  src/uav_sim/
    dynamics.py # phase 1
    integrate.py # phase 1
    aero.py, propulsion.py # phase 2
    atmosphere.py # phase 2
    trim.py, linearise.py # phase 3
    control/pid.py, baseline.py # phase 4
    control/ndi.py, allocation.py # phase 6
    hardware/actuator.py, sensors.py # phase 5
    environment/wind.py # phase 5
    estimation/ekf.py # phase 5
    optimise/objective.py, run_cmaes.py # phase 7
    models/gp_residual.py # phase 8
    montecarlo/sampling.py, campaign.py # phase 9
    analysis/statistics.py, figures.py # phase 10
  tests/
    test_rigid_body.py # the phase 1 gate
    test_aero_signs.py # the phase 2 gate
    test_trim_modes.py # the phase 3 gate
    ...
  scripts/ # every figure has a script here
  results/ # gitignored; regenerated by campaign
  web/ # phase 10 static site
  Makefile # 'make test', 'make reproduce'

```

B Appendix B — Master gate checklist

Ph.	Status	Gate condition
0	☐	Conventions documented; parameters frozen; repo initialised
1	☐	Free fall, ballistic, quaternion norm, angular momentum, energy
2	☐	Lift monotonic; $C_{m_\alpha} < 0$; damping signs negative; symmetry exact
3	☐	Trim converges across envelope; five modes correct; linear/nonlinear match
4	☐	Baseline flies full mission in perfect world; numbers recorded
5	☐	Degradation table produced; runs bit-reproducible by seed
6	☐	Advanced controller beats baseline fairly; degrades gracefully
7	☐	Both controllers optimised; generalisation gap < 0.2
8	☐	GP improves held-out RMSE; calibrated; confidence map exists
9	☐	$\hat{p}$ converged with CI; McNemar significant; failures inspected

Ph.	Status	Gate condition
10	☐	Report reproducible end-to-end; website renders stored results

C Appendix C — The five research questions, and where each is answered

RQ	Question	Answered in
RQ1	How much does nominal performance deteriorate under uncertainty?	Phase 5 gate table, extended by phase 9
RQ2	Does the robust/nonlinear controller genuinely improve things?	Phase 6, confirmed statistically in phase 9
RQ3	Can optimisation improve performance without overfitting?	Phase 7 generalisation gap Δ
RQ4	Can probabilistic models identify low-confidence regions?	Phase 8 confidence map
RQ5	What probability of satisfactory performance is demonstrable?	Phase 9 $\hat{p}$ with Wilson interval

Physics → Validation → Baseline control → Robustness → Optimisation → ML

In that order. Machine learning comes near the end, not the beginning.